



Integração com serviços legados SOAP, contratos e orientação a serviços

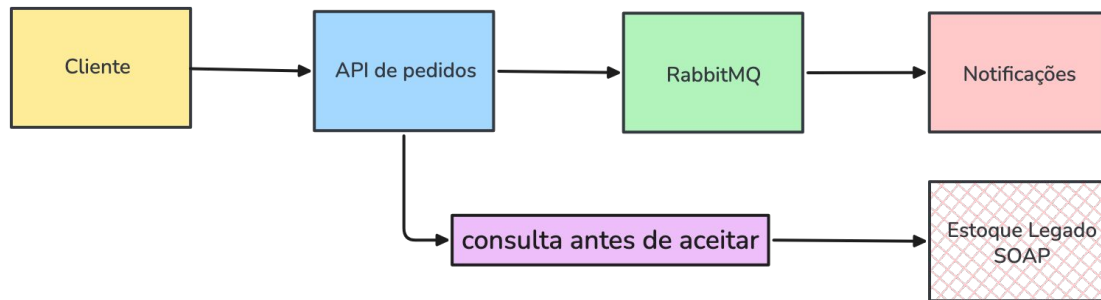
Integração de Sistemas - Aula #07

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Nova regra para aceitar pedidos

- Antes de aceitar um pedido, a API precisa confirmar se a quantidade solicitada está disponível.
- Depois da aceitação, a confirmação ao cliente continua sendo processada por mensageria.
- Mas em que ponto do fluxo a consulta ao estoque precisa ocorrer?

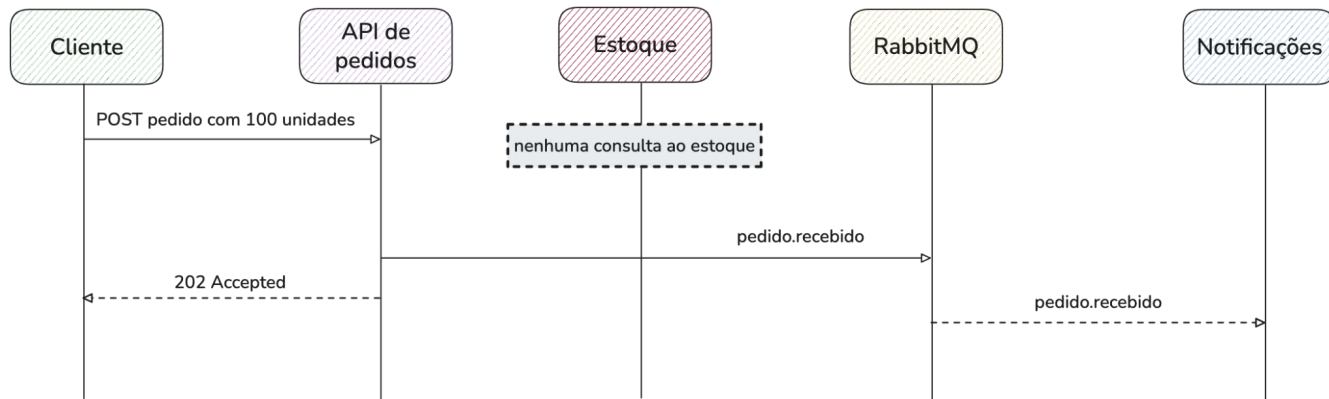


Pedido aceito sem estoque

- Inicie o sistema:
 - `docker compose up -d --build`
- No Swagger, envie:
 - `{"cliente": "Ana", "email": "ana@email.com", "produto": "NOTEBOOK", "quantidade": 100}`
- Retorno:
 - 202 Accepted, `{"id": 1, "status": "recebido"}`
- O pedido de 100 unidades foi aceito... mas o estoque disponível é de apenas 5 unidades.

Fluxo atual do pedido

- A mensageria entrega o evento para o serviço de notificações
- A API ainda não possui a informação necessária para validar o pedido



Contrato do estoque legado

- Inicie somente o serviço de estoque:
 - `docker compose up -d --build estoque-legado`
- Abra no navegador:
 - `http://localhost:8001/?wsdl`
- No documento, procure:
 - A operação `consultarEstoque`
 - Os dados de entrada `sku` e `quantidade`
 - Os dados devolvidos pelo serviço.

"Web Services Description Language (WSDL) é um documento em formato XML que funciona como um contrato ou manual de instruções para descrever um Web Service baseado em SOAP"

Primeira consulta ao estoque

- Crie `cliente_estoque.py`:

```
from zeep import Client
from zeep.helpers import serialize_object

cliente = Client("http://estoque-legado:8000/?wsdl")
resposta = cliente.service.consultarEstoque(sku="NOTEBOOK", quantidade=2)

print(serialize_object(resposta))
```

- Execute dentro da rede do Compose:
 - `docker compose run --rm pedidos python cliente_estoque.py`
- Primeiro resultado:
 - `{"disponivel": True, "saldo": 5}`
- Altere a quantidade para 20 e execute novamente:
 - `{"disponivel": False, "saldo": 5}`
- O cliente Python conseguiu consultar um serviço que não recebe JSON, mas como?

Mensagem SOAP observada

- Exiba os registros do serviço:
 - `docker compose logs --tail 30 estoque-legado`

Delimita a mensagem SOAP

Contém a operação e seus dados

Operação definida pelo contrato

```
<soap-env:Envelope
  xmlns:soap-env="http://schemas.xmlsoap.org/soap/envelope/">
  <soap-env:Body>
    <ns0:consultarEstoque>
      <ns0:sku>NOTEBOOK</ns0:sku>
      <ns0:quantidade>20</ns0:quantidade>
    </ns0:consultarEstoque>
  </soap-env:Body>
</soap-env:Envelope>
```

```
<soap-env:Body>
  <ns0:consultarEstoqueResponse>
    <ns0:disponivel>>false</ns0:disponivel>
    <ns0:saldo>5</ns0:saldo>
  </ns0:consultarEstoqueResponse>
</soap-env:Body>
```

Elementos da chamada SOAP

- **WSDL** descreve a operação, os tipos e o endereço
- **Zeep** é uma biblioteca que transforma o contrato em uma chamada Python
- **SOAP** define a estrutura da mensagem
- **XML** representou os dados
- O **HTTP** transporta a requisição e a resposta



Do experimento para a API

- No `compose.yaml`, informe onde está o contrato e implemente `cliente_estoque.py`

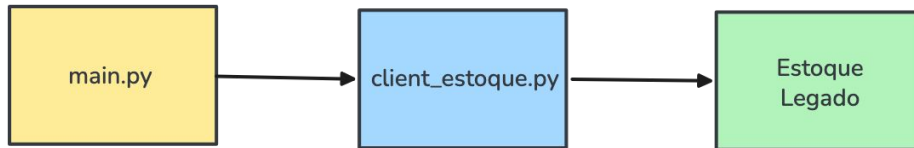
```
pedidos:
  environment:
    ESTOQUE_WSDL: >
      http://estoque-legado:8000/?wsdl
```

```
import os
from zeep import Client

ESTOQUE_WSDL = os.environ["ESTOQUE_WSDL"]

def consultar_estoque(produto: str, quantidade: int):
    cliente = Client(ESTOQUE_WSDL)

    return cliente.service.consultarEstoque(
        sku=produto,
        quantidade=quantidade
    )
```



- Dentro da rede do Compose, `estoque-legado` identifica o contêiner do serviço.

Validação do pedido

- No início de `criar_pedido`:

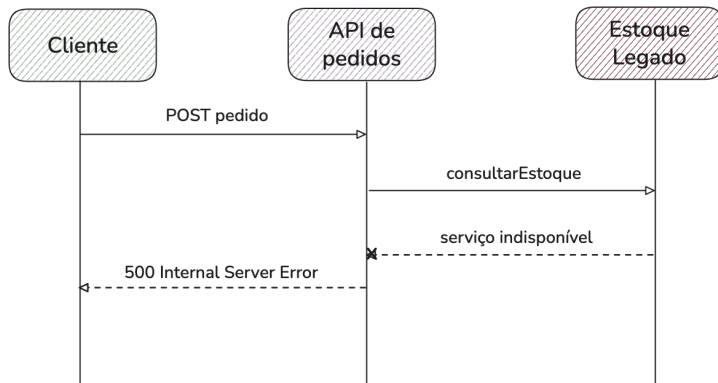
```
from fastapi import HTTPException

@app.post("/pedidos", status_code=202)
def criar_pedido(entrada: PedidoEntrada):
    estoque = consultar_estoque(
        entrada.produto,
        entrada.quantidade
    )

    if not estoque.disponivel:
        raise HTTPException(
            status_code=409,
            detail={
                "mensagem": "Estoque insuficiente",
                "saldo": estoque.saldo
            }
        )
    ...
```

- A consulta ocorre antes do registro do pedido
- Somente pedidos que podem ser atendidos continuam no fluxo existente

Dependências da consulta ao estoque



Dependência	Evidência Observada
Disponibilidade	A API falhou quando o estoque parou
Contrato	O cliente conhece consultarEstoque, sku e quantidade
Tecnologia	O módulo de integração utiliza WSDL, SOAP e Zeep

Exercício

- Aplique o Padrão de Projetos Adapter à nossa aplicação

```
estoque-adapter/  
├── Dockerfile  
├── main.py  
└── requirements.txt
```

```
fastapi  
uvicorn[standard]  
zeep==4.3.3
```

```
FROM python:3.12-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN python -m pip install --no-cache-dir -r requirements.txt  
COPY main.py .  
EXPOSE 8000  
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
import os  
  
from fastapi import FastAPI, Query  
from zeep import Client  
  
app = FastAPI(title="Adaptador de estoque")  
  
cliente_soap = Client(  
    os.environ["ESTOQUE_WSDL"]  
)  
  
@app.get("/estoques/{sku}")  
def consultar_estoque(  
    sku: str,  
    quantidade: int = Query(gt=1)  
):  
    resposta = cliente_soap.service.consultarEstoque(  
        sku=sku,  
        quantidade=quantidade  
    )  
  
    return {  
        "sku": sku,  
        "quantidade": quantidade,  
        "disponivel": resposta.disponivel,  
        "saldo": resposta.saldo  
    }
```

```
estoque-adapter:  
build: ./estoque-adapter  
ports:  
- "8002:8000"  
environment:  
    ESTOQUE_WSDL: >  
        http://estoque-legado:8000/?wsdl  
depends_on:  
    estoque-legado:  
        condition: service_healthy
```

Exercício

- Inicie o adaptador:
 - `docker compose up -d --build pedidos estoque-adapter`
- Abra
 - `http://localhost:8002/docs`
- Teste
 - `GET /estoques/NOTEBOOK?quantidade=2`
- Depois
 - `GET /estoques/NOTEBOOK?quantidade=20`
- O adaptador devolve JSON nos dois casos.
- Estoque insuficiente representa um resultado válido da consulta.

API usando o adaptador

- No serviço pedidos, substitua ESTOQUE_WSDL por:
 - environment:
 - ESTOQUE_URL: `http://estoque-adapter:8000`
- Remova o código do Zeep de pedidos/main.py e ajuste a verificação
- Reconstrua e teste
 - `docker compose up -d --build pedidos estoque-adapter`

```
import httpx

ESTOQUE_URL = os.environ["ESTOQUE_URL"]

def consultar_estoque(
    produto: str,
    quantidade: int
):
    resposta = httpx.get(
        f"{ESTOQUE_URL}/estoques/{produto}",
        params={"quantidade": quantidade},
        timeout=3
    )
    resposta.raise_for_status()
    return resposta.json()
```

```
if not estoque["disponivel"]:
    raise HTTPException(
        status_code=409,
        detail={
            "mensagem": "Estoque insuficiente",
            "saldo": estoque["saldo"]
        }
    )
```

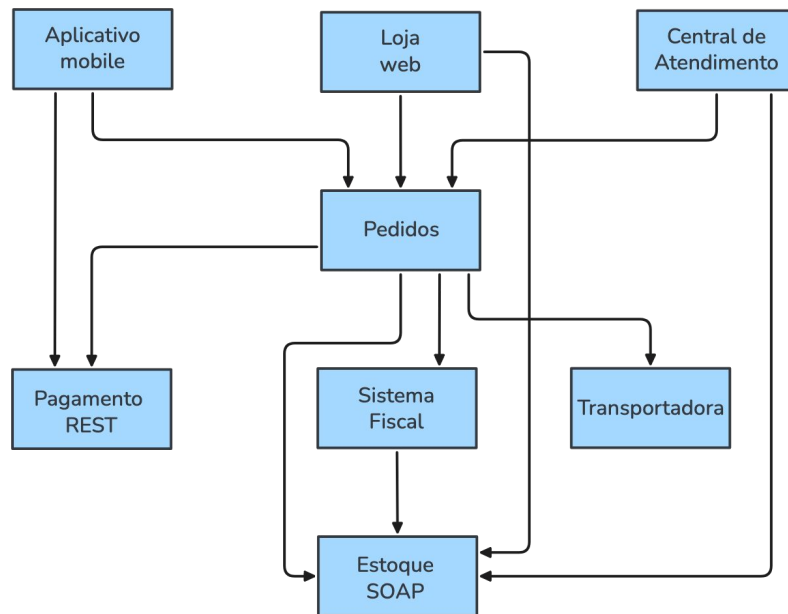
Processo completo do pedido

- A verificação de estoque representa apenas uma etapa
- Cada etapa pode depender de um sistema diferente:
 - Estoque legado
 - Operadora de pagamento
 - Transportadora
 - Sistema fiscal
 - Serviço de notificações.



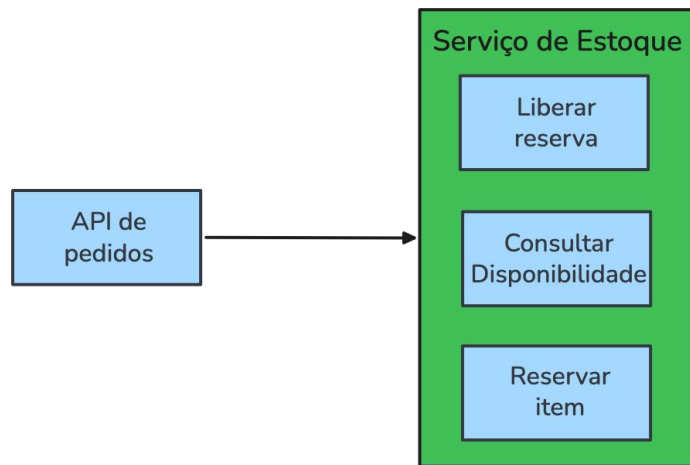
Malha de integrações ponto a ponto

- Cada linha representa um contrato, um formato de dados e uma possibilidade de falha que algum sistema precisa tratar.
- Se o serviço de estoque alterar sua operação, quantos consumidores precisarão mudar?



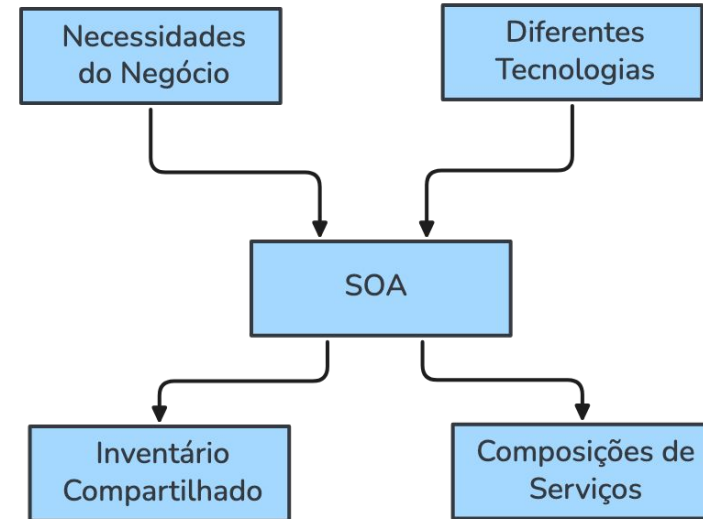
Serviço como conjunto de capacidades

- Um serviço publica um contrato para um conjunto relacionado de capacidades
- A API de pedidos é um consumidor, enquanto que o serviço de estoque expõe um conjunto de capacidades para seu uso e gerencia o estoque disponível.



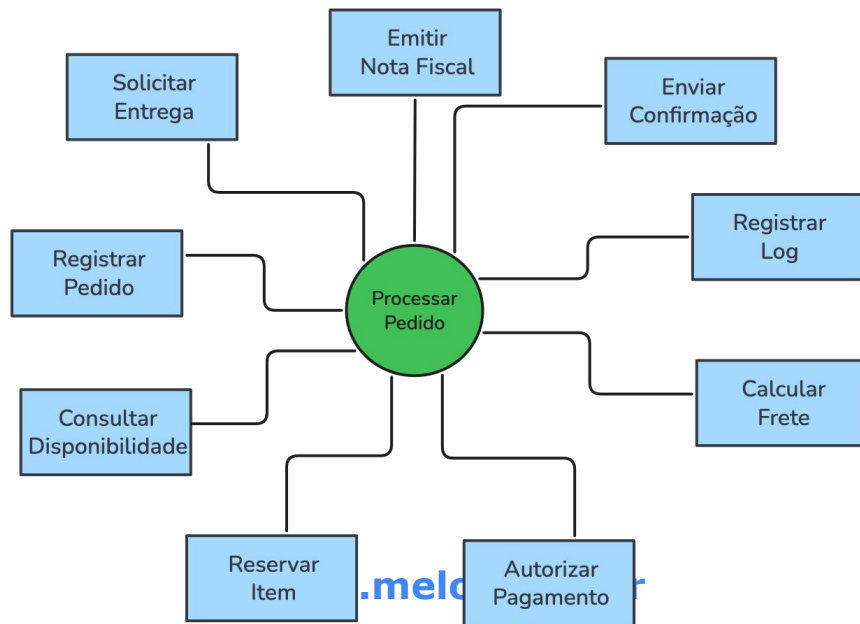
Arquitetura Orientada a Serviços

- SOA organiza a automação de uma organização por meio de serviços com contratos publicados, reunidos em inventários e utilizados em composições.
- Outras características da SOA:
 - Orientada pelo negócio
 - Neutra em relação a fornecedores
 - Abrangência organizacional e
 - Centrada em composições



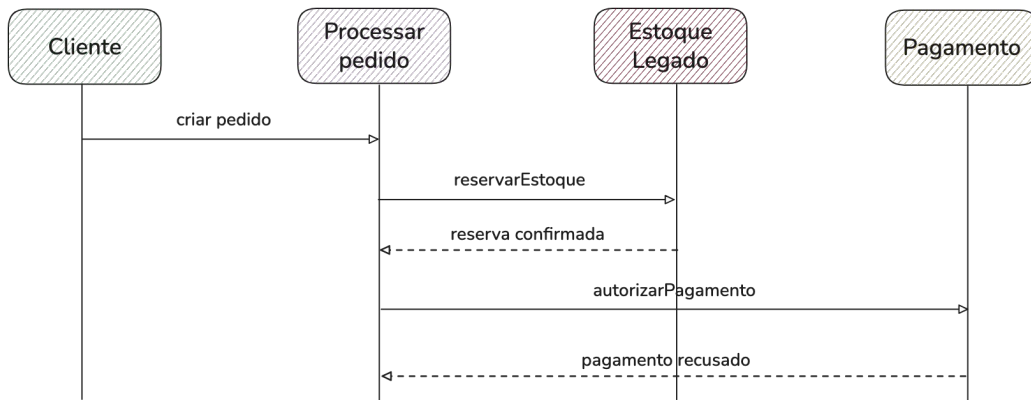
Exercício

- Para o projeto da disciplina liste:
 - O nome dos serviços que sua equipe desenvolverá
 - Quem precisa consumir essas capacidades
- Exemplo para *pedido*:



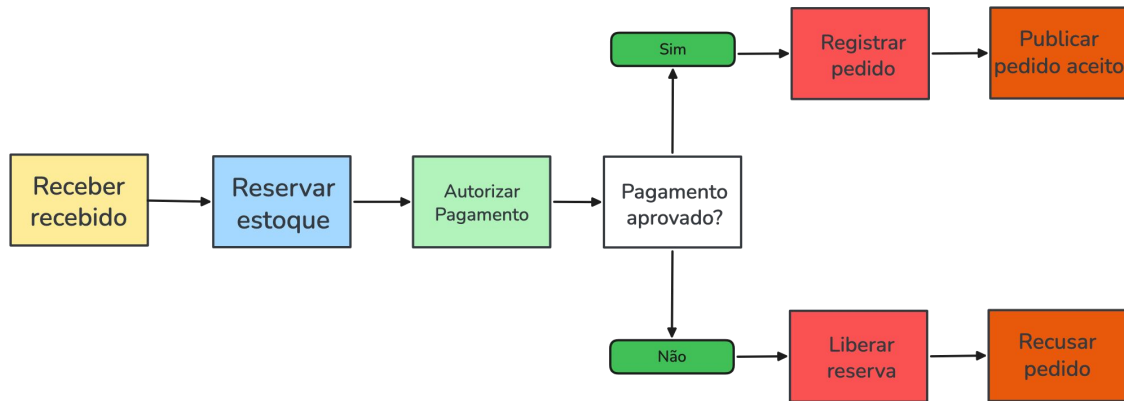
Falha no meio do processo

- Para analisar a coordenação, suponha que o serviço de estoque também ofereça `reservarEstoque` e `liberarReserva`.
- Uma unidade continua reservada, mas o pedido não pode prosseguir.
- Quem deve perceber a falha e solicitar a liberação do estoque?



Orquestração do pedido

- Na orquestração, um componente conhece as etapas do processo, decide a próxima chamada e trata os resultados.
- No exemplo, Processar pedido coordena o fluxo.
- Estoque e pagamento executam suas próprias operações sem conhecer o processo completo.
- Liberar o estoque desfaz o efeito de uma etapa já concluída. Essa ação funciona como uma **compensação**.



BPEL como processo executável

- Business Process Execution Language (BPEL) é uma linguagem baseada em XML para descrever a coordenação de serviços
- Exemplo

```
<sequence>
  <invoke
    partnerLink="estoque"
    operation="reservarEstoque" />

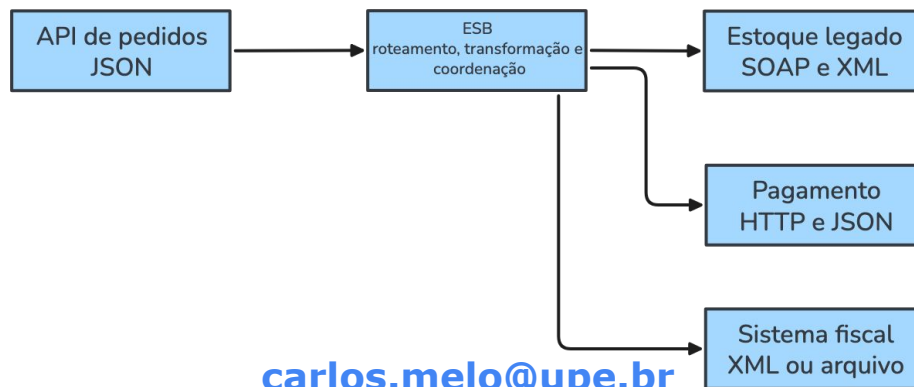
  <invoke
    partnerLink="pagamento"
    operation="autorizarPagamento" />

  <if>
    <condition>
      $pagamento/aprovado = false()
    </condition>

    <invoke
      partnerLink="estoque"
      operation="liberarReserva" />
  </if>
</sequence>
```

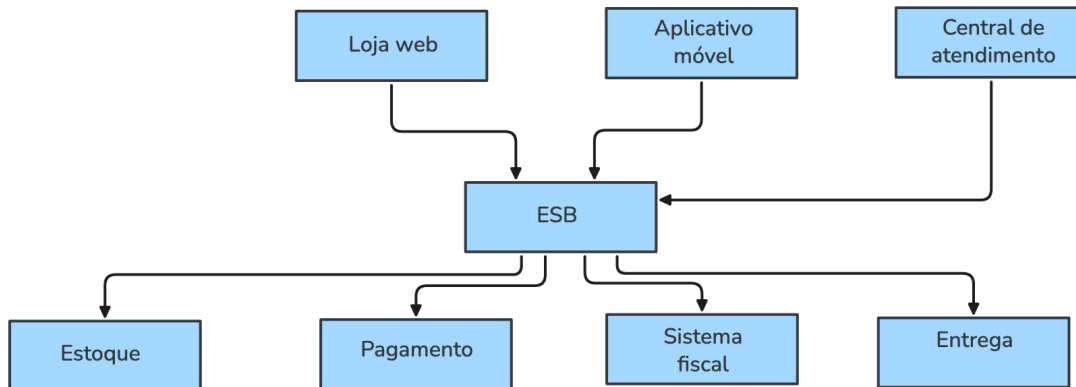
Do Adapter ao ESB

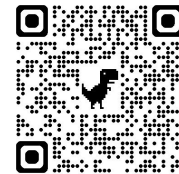
- O estoque-adapter resolve uma incompatibilidade específica.
- Um ESB pode reunir vários adapters e realizar mediação para diferentes aplicações.
- O ESB, é um *middleware* pelo qual diferentes sistemas podem trocar mensagens mesmo utilizando contratos, formatos ou protocolos distintos e serve como infraestrutura de integração.
- Ele é opcional em SOA e não deve assumir as regras de negócio dos serviços.
 - Conversões e roteamento podem ficar no ESB.
 - Regras de estoque e pagamento permanecem nos respectivos serviços.



Risco de uma SOA accidental

- Quando o ESB passa a concentrar todas as chamadas, regras e decisões, qualquer mudança atravessa o componente central, gerando excesso de centralização:
 - Regras de estoque ou pagamento implementadas no barramento
 - Toda entrega exige alteração e teste do ESB
 - Uma falha no barramento interrompe diversos processos.
- E se para pagamentos acima de R\$ 5.000 exigissem uma análise adicional, essa regra pertence ao ESB ou ao serviço de pagamento?





Prof. Dr. Carlos Melo

<https://calendly.com/prof-casm>

 carlos.melo@upe.br